

```

const express = require('express');
const fs = require('fs');
const path = require('path');
const puppeteer = require('puppeteer');
const cheerio = require('cheerio');

const app = express();
app.use(express.json());
app.use(express.static(__dirname));

const DB_PATH = path.join(__dirname, 'db.json');
const BACKUP_PATH = path.join(__dirname, 'db_backup.json');
const LOG_PATH = path.join(__dirname, 'debug.log');

// Функція запису повідомлень у лог-файл
function logToFile(message) {
    const timestamp = new Date().toISOString();
    fs.appendFileSync(LOG_PATH, `[${timestamp}] ${message}\n`, 'utf8');
}

function readDB() {
    const emptyStructure = { vacancies: [] };
    if (!fs.existsSync(DB_PATH)) {
        if (fs.existsSync(BACKUP_PATH)) {
            fs.copyFileSync(BACKUP_PATH, DB_PATH);
        } else {
            fs.writeFileSync(DB_PATH, JSON.stringify(emptyStructure, null, 2), 'utf8');
            return emptyStructure;
        }
    }
}

```

```

    try {
        return JSON.parse(fs.readFileSync(DB_PATH, 'utf8'));
    } catch (e) {
        fs.writeFileSync(DB_PATH, JSON.stringify(emptyStructure, null, 2), 'utf8');
        return emptyStructure;
    }
}

```

```

function writeDB(data) {
    if (fs.existsSync(DB_PATH)) {
        fs.copyFileSync(DB_PATH, BACKUP_PATH);
    }
    fs.writeFileSync(DB_PATH, JSON.stringify(data, null, 2), 'utf8');
}

```

```

app.get('/api/vacancies', (req, res) => {
    res.json(readDB().vacancies);
});

```

```

app.post('/api/vacancies/status', (req, res) => {
    const { url, status } = req.body;
    const db = readDB();
    const vac = db.vacancies.find(v => v.url === url);
    if (vac) {
        vac.status = status;
        writeDB(db);
        return res.json({ success: true });
    }
    res.status(404).json({ error: "Вакансію не знайдено" });
});

```

```
// Швидкий API-парсер через AJAX з логуванням структури відповіді
app.get('/api/scrape', async (req, res) => {

  // Очищуємо лог-файл перед кожним новим скануванням
  fs.writeFileSync(LOG_PATH, '', 'utf8');
  logToFile('=== ЗАПУСК НОВОГО АНАЛІЗУ API ===');

  let browser;

  try {
    browser = await puppeteer.launch({
      headless: true,
      args: ['--disable-blink-features=AutomationControlled']
    });

    const page = await browser.newPage();

    await page.setUserAgent('Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
    like Gecko) Chrome/124.0.0.0 Safari/537.36');

    let allScrapedUrls = [];

    let pageNum = 1;

    let offset = 0;

    let hasMoreData = true;

    console.log('Розпочинаю швидке завантаження через AJAX API Lobby X...');
    logToFile('Ініціалізація циклу проходження сторінок API.');
```

```
    while (hasMoreData) {
      const queryParams = new URLSearchParams({
        action: 'loadmore',
        offset: offset.toString(),
```

```
    page: pageNum.toString(),
    category_sphere: '27',
    language: 'uk'
  });

const ajaxUrl = 'https://lobbyx.army?' + queryParams.toString();

console.log(`Завантажую порцію даних: Сторінка ${pageNum} (Зміщення offset: ${offset})...`);
logToFile(`Запит на лінк: ${ajaxUrl}`);

await page.goto(ajaxUrl, { waitUntil: 'networkidle2' });

// Витягуємо сирий текст відповіді, що прийшов від сервера
const rawContent = await page.evaluate(() => document.body.innerText);

logToFile(`--- СИРИЙ ВМІСТ СЕРВЕРА (СТОРІНКА ${pageNum}) ---`);
logToFile(rawContent.substring(0, 2000)); // Логуємо перші 2000 символів структури запиту
logToFile(`--- КІНЕЦЬ СИРОГО ВМІСТУ СТОРІНКИ ${pageNum} ---`);

const $ = cheerio.load(rawContent);
const pageCards = $('a.job-item');

logToFile(`Кількість виявлених карток a.job-item через Cheerio: ${pageCards.length}`);

if (pageCards.length === 0) {
  console.log('Нових карток немає. Викачування списку завершено.');
  logToFile('Аналіз завершено: отримано 0 карток контенту.');
  hasMoreData = false;
  break;
}
```

```
pageCards.each((index, element) => {  
  const href = $(element).attr('href');  
  if (href && !allScrapedUrls.includes(href)) {  
    allScrapedUrls.push(href);  
  }  
});
```

```
offset += pageCards.length;
```

```
pageNum++;
```

```
if (pageNum > 50) {  
  logToFile('Переривання: Досягнуто ліміту безпеки у 50 сторінок.');
```

break;

```
}  
await new Promise(r => setTimeout(r, 500));  
}
```

```
console.log(`Всього через API знайдено унікальних IT-вакансій: ${allScrapedUrls.length}. Починаю  
глибокий збір заголовків...`);
```

```
logToFile(`Унікальних URL адрес у масиві для обходу: ${allScrapedUrls.length}`);
```

```
const db = readDB();
```

```
let addedCount = 0;
```

```
for (const url of allScrapedUrls) {  
  const isExist = db.vacancies.some(v => v.url === url);  
  if (isExist) continue;  
  
  try {  
    logToFile(`Перехід на внутрішню сторінку: ${url}`);
```

```

await page.goto(url, { waitUntil: 'networkidle2' });

await page.waitForSelector('h1', { timeout: 6000 }).catch(() => {});

const details = await page.evaluate(() => {

    const h1Element = document.querySelector('h1.vacancy-name') ||
document.querySelector('h1');

    let title = h1Element ? h1Element.innerText.trim() : '';

    const h4Elements = Array.from(document.querySelectorAll('h4'));

    let unit = 'Сили Оборони України';

    if (h4Elements.length > 0) {

        unit = h4Elements[0].innerText.trim();

    }

    if (unit.toLowerCase().includes('обов') || unit.toLowerCase().includes('вимог') ||
unit.toLowerCase().includes('умов')) {

        unit = 'Військова частина / Підрозділ';

    }

    return { title, unit };

});

if (details.title && details.title.length > 2) {

    db.vacancies.push({

        url: url,

        title: details.title,

        unit: details.unit,

        status: 'new'

    });

```

```

        addedCount++;

        writeDB(db);

        console.log(`[ДОДАНО] ${details.unit} -> ${details.title}`);

        logToFile(`База оновлена: ${details.unit} -> ${details.title}`);
    }

    await new Promise(r => setTimeout(r, 800));

    } catch (e) {
        console.log(`Помилка сторінки ${url}:`, e.message);

        logToFile(`Помилка парсингу URL ${url}: ${e.message}`);
    }
}

await browser.close();

console.log(`Процес повністю завершено. До локальної бази успішно додано: ${addedCount}`);

logToFile(`=== РОБОТУ ЗАВЕРШЕНО. Успішно внесено позицій: ${addedCount} ===`);

res.json({ success: true, added: addedCount });

} catch (err) {
    console.error('Помилка парсингу:', err);

    logToFile(`КРИТИЧНА ПОМИЛКА СКРИПТА: ${err.message}`);

    if (browser) await browser.close();

    res.status(500).json({ error: err.message, added: 0 });
}

});

app.listen(3000, () => console.log('Сервер працює: http://localhost:3000'));

```